01 QUESTION ONE · COMPONENT CLUSTERING

How clustering factor improves and helps in COM programming?

4 A4 PAGES
C++ PROGRAM ON PAGE 4

INTRODUCTION TO COM

COM (Component Object Model) is a Microsoft specification, introduced in **1993**, describing how software components are built and how they talk to one another. A *component* is a ready-made piece of software that performs one clear job and can be plugged into different programs without being recompiled. Before COM, reusable code was shared through ordinary DLLs, but this created serious trouble — no fixed way to ask a DLL what it could do, C and C++ compilers mangling function names differently, and a new DLL often breaking an older program. This was nicknamed **"DLL Hell"**.

COM solved this with a **binary standard**. A component written in C++, Visual Basic or Delphi can be used by a program written in a different language, because the two agree only on the layout of a virtual table, not on the source code. A component exposes its services through **interfaces**, and every interface derives from `IUnknown`, which provides `QueryInterface`, `AddRef` and `Release`. Each component is identified by a globally unique **GUID** — a CLSID for a class, an IID for an interface — so two programmers anywhere in the world can never clash over a name.

Depending on where the component runs, a COM call may be **in-process** (same program), **out-of-process** (another program on the same machine) or **remote** (another machine — this is **DCOM**). The further apart two components are, the more expensive the call becomes, and this is exactly where the clustering factor matters.



FIG. 1 — THE COM CALL COST LADDER. A HIGHER CLUSTERING FACTOR KEEPS THE CHATTY OBJECTS ON THE LEFT, WHERE EACH CALL IS CHEAPEST.

WHAT IS THE CLUSTERING FACTOR?

Clustering means keeping related things together so they can work as one group. In COM, the **clustering factor** measures how well the objects that talk most often have been kept together — in the same process, or at least the same machine — so their calls stay **local**. A high factor means the communication happens in one place; a low factor means the components are scattered and every call crosses a process or network boundary.

The reason this matters is that the cost of a COM call is not the same everywhere. A call inside the same process is a direct jump through a v-table and takes nanoseconds. A call crossing an **apartment** boundary needs **marshalling** — a proxy packs the arguments, a stub unpacks them, the call runs, and the result is packed back. A call leaving the process entirely adds context switches, and for DCOM a full network round-trip, so the cost grows from nanoseconds to microseconds to milliseconds as you move outward. Think of a kitchen: if every ingredient for a dish is on the same counter, cooking is fast; if the salt is in the kitchen, the onions in the store-room and the water in the garden, the cook spends more time walking than cooking.

A designer therefore improves the factor in two steps. First find the objects that exchange messages most often — the “chatty” parts of the system. Then bring those objects into the same **process and the same apartment**, so their frequent conversation never leaves one boundary. Rarely-used, very heavy objects are better kept out-of-process so they do not occupy memory in the main program. A second benefit is **thread safety**: every apartment follows its own threading rule, so related objects sharing one apartment share one rule, removing a whole class of bugs where one thread changes data while another reads it.

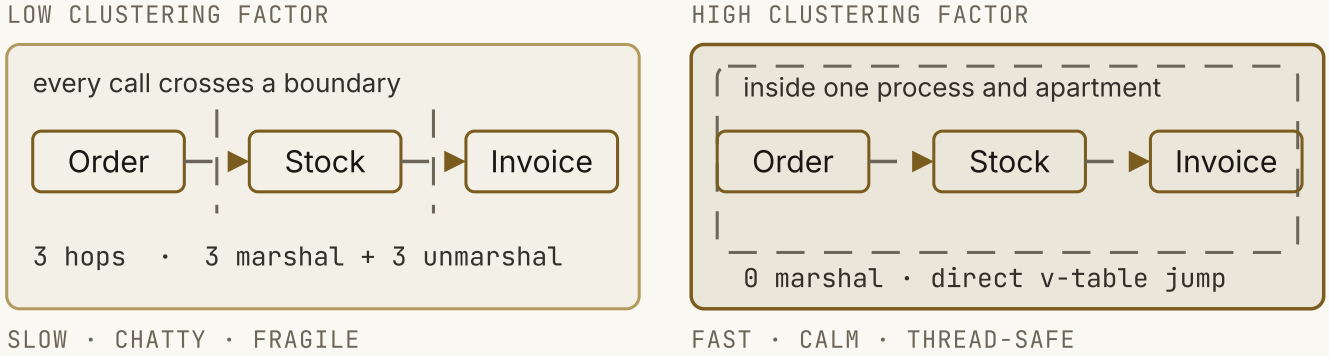


FIG. 2 — THE SAME THREE OBJECTS, ARRANGED TWO WAYS. ON THE LEFT EVERY MESSAGE CROSSES A BOUNDARY AND MUST BE MARSHALLED; ON THE RIGHT THE FREQUENTLY-TALKING OBJECTS LIVE INSIDE ONE BOUNDARY, SO THE CALLS TURN INTO ORDINARY LOCAL JUMPS.

HOW THE CLUSTERING FACTOR IMPROVES COM PROGRAMMING

A well-chosen clustering factor improves a COM system in five concrete ways. **First, it removes marshalling overhead:** objects that exchange data most frequently live in the same process, so the proxy–stub layer is skipped and both CPU cost and call delay fall sharply. **Second, it improves throughput and response time:** a business object that calls a data object a thousand times per transaction becomes noticeably faster when the two are co-located. **Third, it improves CPU cache locality**, because the code and data of closely related objects stay warm in the processor cache. **Fourth, it simplifies apartment and thread management,** since objects sharing a component share one apartment and one threading rule. **Fifth, it scales and stays reliable,** because a cluster of servers can share the load and keep working even if one node fails.

The table below shows the practical difference between a loosely-packed and a tightly-packed COM application.

POINT OF COMPARISON	WITHOUT CLUSTERING (LOW FACTOR)	WITH CLUSTERING (HIGH FACTOR)
Call path	Cross-process / cross-machine	In-process (direct v-table)
Marshalling cost	High — data packed and copied every call	Almost zero
Latency per call	Microseconds to milliseconds	A few nanoseconds
Failure handling	One failure stops the chain	Other cluster nodes take over
Load balancing	Not possible	Requests spread over nodes

In real life, a bank's COM transaction system keeps the account, ledger and validation objects together inside one clustered business tier. Because these three talk on almost every transaction, clustering them means a transaction hits the fast local path many times and only the final write touches the database — quicker and more stable under load.

C++ PROGRAM

THE PROGRAM BELOW BUILDS A SMALL CLUSTER OF COM-STYLE COMPONENTS AND SHOWS THAT THE CALLS BETWEEN THEM STAY IN ONE PLACE (LOCAL, NO MARSHALLING). COPY THIS PROGRAM ON PAGE 4.

CLUSTER_DEMO.CPP

```cpp
// cluster_demo.cpp - grouping related components keeps calls local
#include <iostream>
#include <string>
using namespace std;

class IComponent {                        // COM-style interface
public:
    virtual void execute(const string& t) = 0;
    virtual ~IComponent() {}
};
class AccountService : public IComponent {          // cluster member 1
public:
    void execute(const string& t) override { cout << "[Account] " << t << endl; }
};
class LedgerService : public IComponent {           // cluster member 2
public:
    void execute(const string& t) override { cout << "[Ledger ] " << t << endl; }
};
class Cluster {                           // keeps related objects together
    AccountService account;               // local member  →  fast path
    LedgerService  ledger;                // local member  →  fast path
public:
    void process() {
        account.execute("debit Rs.2500");            // stays in-process
        ledger.execute ("record the debit entry");   // stays in-process
        cout << "Done locally - no marshalling." << endl;
    }
};
int main() {
    Cluster bank;                         // all objects live together
    bank.process();
    return 0;
}
```

Output of the Program

OUTPUT

```
[Account] debit Rs.2500
[Ledger ] record the debit entry
Done locally - no marshalling.
```

CONCLUSION

The clustering factor is a way of measuring how tightly the related components of a COM application are packed together. When the objects that talk most often share a process, their calls stay local, marshalling disappears, and both speed and reliability improve. In short, a good clustering factor is one of the simplest and cheapest ways to make a COM system faster and stronger.

02 QUESTION TWO · PERSISTENCE

Explain the meaning of persistence in COM / DCOM.

4 A4 PAGES
C++ PROGRAM ON PAGE 4

MEANING OF PERSISTENCE IN COM / DCOM

Persistence means the ability of an object to save its **state** into a permanent store and restore it later, even after the program has closed or the computer been switched off. The *state* of an object is simply the values held in its data members — for a bank object, the account number, the holder's name and the current balance. Memory (RAM) is **volatile**: the moment a process ends, everything it held is wiped out. Persistence is what lets an object survive that moment.

In COM, such an object can write itself into a **stream** or a **storage** — a place where bytes are kept safely — and read itself back later. Turning a live object into bytes is called **serialisation**; rebuilding it from those bytes is called **deserialisation**. A video game shows this clearly: while you play, the game keeps your score and level only in memory, so a power cut loses everything. *Save Game* writes exactly those values to a file on disk, and *Load Game* reads them back so you continue where you left off — which is precisely what a persistent COM object does.

Persistence is not the same as storage or serialisation, though all three are related. **Storage** is the place where bytes are kept. **Serialisation** is the act of converting between the object and those bytes. **Persistence** is the larger promise that the state will still be there tomorrow — possible only when the two work together. A COM object with persistence can be saved today, its process closed, the machine restarted, and the very same object brought back to life with all its data intact.

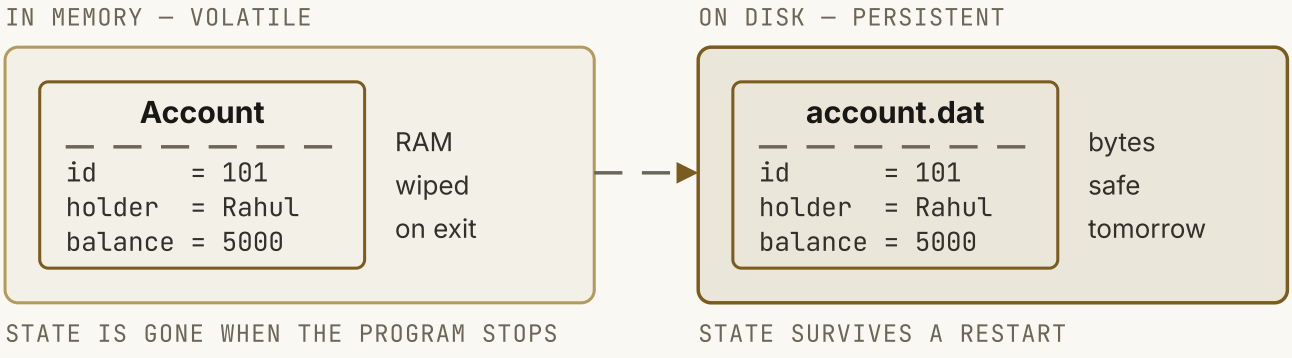


FIG. 1 — THE STATE OF ONE OBJECT, SEEN TWICE. ON THE LEFT IT LIVES ONLY IN VOLATILE RAM AND DIES WITH THE PROCESS; ON THE RIGHT THE SAME VALUES HAVE BEEN WRITTEN INTO A FILE, SO THEY CAN BE READ BACK TOMORROW AND THE OBJECT REBUILT AS IT WAS.

WHY PERSISTENCE IS NEEDED

Persistence is needed for the most practical of reasons. A program is not expected to run for ever — the user closes it, the machine restarts, and sometimes a crash occurs. Without persistence every useful object would vanish on each shutdown. Persistence lets a document, a customer record, a user's preferences or a partly-finished transaction live safely between runs. It also allows data to be **shared** (one program saves an object, another opens it later) and lets a system **recover** after a failure, because the last saved state is still on the disk.

HOW COM SUPPORTS PERSISTENCE

COM offers two approaches. In **data-centric persistence** the component saves only its plain data and must know a fixed file layout. In **object-centric persistence** the object saves and restores its own internal state through standard interfaces, so the client need not know how the object is built. The second is cleaner, and COM supports it through `IPersistStream` and `IPersistStorage`.

The storage itself comes from **Structured Storage**, a "file system inside a single file", better known as a **compound file**. Inside it, **storages** work like folders and **streams** like files, managed through `IStorage` and `IStream`. A **moniker** is a special name that identifies a persisted object and lets any client find and re-activate it later, which is what allows a saved object to be linked rather than merely copied. The two interfaces are chosen by size: a small object needs only a flat `IStream`, while a large object made of many parts uses `IStorage`, which holds one stream per part so a client can open one part of the file without reading the whole thing.

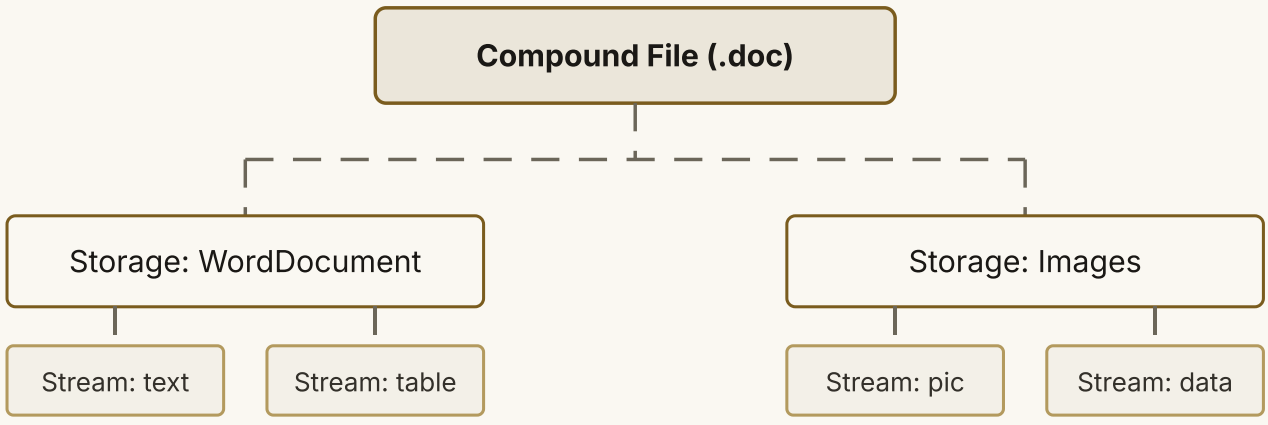


FIG. 2 — STRUCTURED STORAGE. A COMPOUND FILE BEHAVES LIKE A SMALL FILE SYSTEM: STORAGES ACT AS FOLDERS AND STREAMS AS FILES, ALL INSIDE A SINGLE ORDINARY FILE.

PERSISTENCE IN DCOM (DISTRIBUTED COM)

DCOM is COM extended across a network, so a component running on one machine can be used by a program on another. Here persistence takes on an extra duty: not only must the object save its state, its **location** must survive too. When a client wants a remote object that is not currently running, DCOM uses the persisted reference to start it on the correct machine (remote activation) and then reconnect. This lets an idle server object be "put to sleep" and brought back when needed, saving a great deal of memory across a large network. In practice this is achieved through a handful of well-known mechanisms, summarised below.

MECHANISM	WHAT IS STORED	TYPICAL EXAMPLE
Stream (`IStream`)	A flat sequence of bytes	An image or a small record
Storage (`IStorage`)	Storages and streams (compound file)	A Word document (.doc)
Moniker	A name that locates an object	A linked spreadsheet cell
Registry	Configuration and class registration	Component settings
Database	Business data fetched on demand	An ADO recordset

Everyday software shows persistence constantly. Saving a Microsoft Word file writes its document object into a compound file. Changing a setting makes Windows store it in the registry. A web application keeping your session on the server is persisting a COM object between requests.

In a DCOM system persistence is worth even more, because the objects are spread over many machines. If a remote object had to stay alive for ever, each server would slowly fill with idle objects and the network would become slow. Persistence lets an idle object be shut down and, when a client asks for it again, rebuilt from its saved state on the correct server. Large systems also use it for fault tolerance: if a server stops mid-operation, the last saved state is still available, so the work resumes on another machine from the point where it stopped.

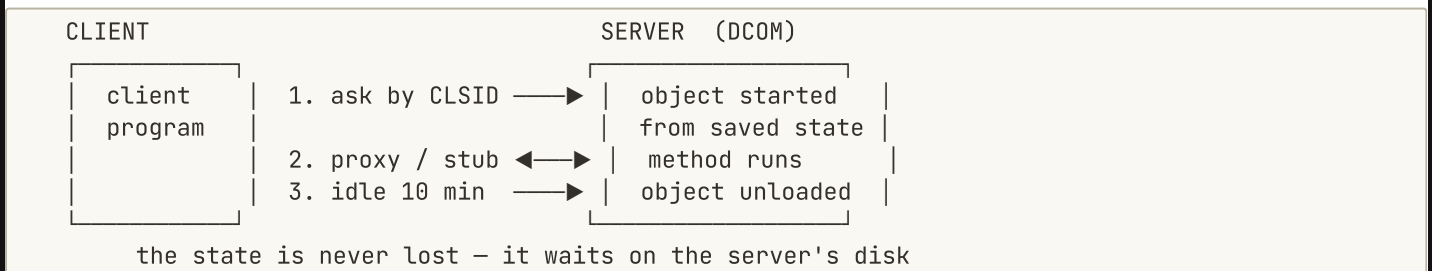


FIG. 3 — PERSISTENCE IN DCOM, DRAWN AS A CONVERSATION. THE CLIENT NEVER KEEPS THE OBJECT ALIVE; IT ASKS FOR IT BY NAME, THE SERVER REBUILDS IT FROM ITS SAVED STATE, AND WHEN THE OBJECT FALLS IDLE IT IS UNLOADED AGAIN — ITS STATE KEPT SAFELY ON DISK.

C++ PROGRAM

THE PROGRAM BELOW SHOWS THE CORE IDEA OF PERSISTENCE — AN OBJECT WRITES ITS STATE TO A FILE, THE OBJECT IS THEN REPLACED, AND THE SAVED STATE IS LOADED BACK. COPY THIS PROGRAM ON PAGE 4.

PERSIST_DEMO.CPP

```cpp
// persist_demo.cpp - saving and restoring an object's state
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Account {                          // a persistable component
    int    id;
    string holder;
    double balance;
public:
    Account(int i, string h, double b) : id(i), holder(h), balance(b) {}
    void save(const string& file) {      // ---- persist to disk ----
        ofstream out(file);
        out << id << "\n" << holder << "\n" << balance << "\n";
        out.close();
        cout << "State saved to " << file << endl;
    }
    void load(const string& file) {      // ---- restore from disk ----
        ifstream in(file);
        in >> id >> holder >> balance;
        in.close();
        cout << "Restored → id=" << id << " holder=" << holder
             << " balance=" << balance << endl;
    }
};
int main() {
    Account acc(101, "Rahul", 5000.0);   // live object in memory
    acc.save("account.dat");             // state written to the disk
    Account fresh(0, "-", 0.0);          // a brand-new empty object
    fresh.load("account.dat");           // state comes back from disk
    return 0;
}
```

Output of the Program

OUTPUT

```
State saved to account.dat
Restored → id=101 holder=Rahul balance=5000
```

CONCLUSION

Persistence in COM and DCOM is the ability of an object to write its state into a permanent store and to bring that state back later through serialisation and deserialisation. Structured Storage provides the streams and storages that hold the data, monikers locate the saved objects, and DCOM even keeps the object's location alive so that remote objects can be re-activated. Without persistence, every COM object would die with its process; with persistence, objects outlive the program that created them.